

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

EMANIP

O QUE VEM POR AÍ?

Nesta aula, aprenderemos sobre Chains no LangChain, que são fluxos de trabalho sequenciais usados para processar consultas em várias etapas. Vamos explorar como criar prompts customizados, como roteá-los com base no conteúdo da pergunta e como combinar diferentes cadeias (chains) para fornecer respostas precisas e adequadas.



HANDS ON

Nesta aula prática, veremos sobre Chains no LangChain, que são estruturas usadas para criar fluxos de trabalho automatizados com modelos de linguagem. Exploraremos como encadear diferentes prompts e modelos de IA para responder perguntas complexas e como roteá-las para especialistas específicos.



SAIBA MAIS

Nesta aula, vamos explorar o que são as Chains, seus tipos e benefícios, além de mostrar exemplos práticos para ajudar a entender como utilizá-las em suas aplicações.

As **Chains** no LangChain são uma forma de conectar diferentes etapas de processamento em sequência. Elas funcionam ligando a saída de uma tarefa como entrada para a próxima, criando um fluxo contínuo de operações. Esse encadeamento permite criar soluções mais complexas e flexíveis, dividindo grandes problemas em pequenas partes que podem ser processadas separadamente.

A figura 1 exemplifica como podemos trabalhar com chains:

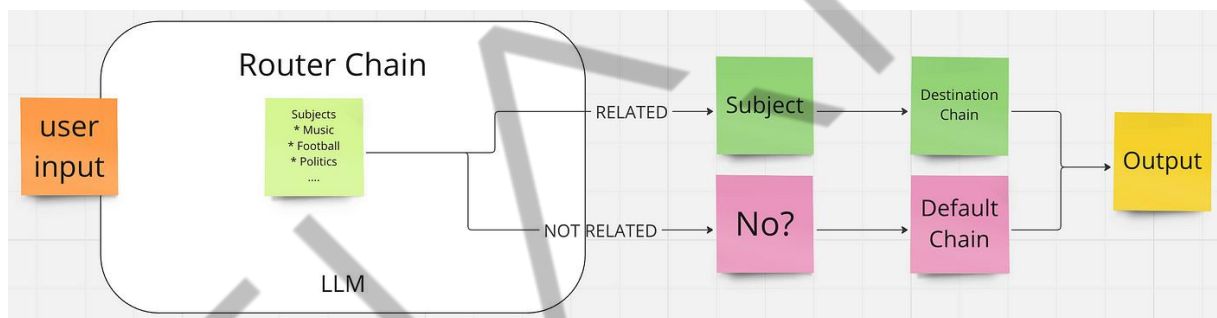


Figura 1 - Fluxo Chains
Fonte: Okan Yenigün

Note que, essencialmente, uma Chain funciona como um pipeline, onde a saída de um prompt serve como input para o próximo, formando uma sequência lógica de eventos. Esse processo pode ser repetido várias vezes, criando fluxos de trabalho robustos que podem lidar com uma grande diversidade de problemas em inteligência artificial.

O LangChain oferece diversos tipos de Chains, cada uma adequada para um tipo específico de fluxo de trabalho. Vejamos algumas delas a seguir:

A **Sequential Chain** é uma das mais comuns e simples. Nesse tipo, os prompts são executados em sequência, onde a saída de uma etapa é usada diretamente como entrada para a próxima. Esse encadeamento linear facilita o processamento de dados que dependem diretamente de respostas anteriores.

Vejamos um exemplo prático:

```
from langchain.llms import OpenAI
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain.chains import SequentialChain
from langchain.memory import SimpleMemory
from datetime import datetime

llm = OpenAI()

template_rapper: str = """Você é um compositor de rap renomado. Sua missão é
criar uma letra de rap
inspirada no tema fornecido.

Tema da música:
{input}\
"""

prompt_template_rapper: PromptTemplate = PromptTemplate(
    input_variables=["input"], template=template_rapper)

cadeia_rapper: LLMChain = LLMChain(
    llm=llm, output_key="letra", prompt=prompt_template_rapper)

template_verificador: str = """Você é responsável por revisar letras de rap. Seu
trabalho é verificar se as letras contêm
algum conteúdo violento ou linguagem inadequada.

Por favor, responda no formato de um dicionário Python:
letra: a letra recebida
Palavras_violentas: Verdadeiro ou Falso
```

Aqui está a letra a ser verificada:

```
{letra}\
```

```
''''''
```

```
prompt_template_verificador: PromptTemplate = PromptTemplate(
    input_variables=["letra"], template=template_verificador)
```

```
cadeia_verificador: LLMChain = LLMChain(
    llm=llm, output_key="letra_verificada", prompt=prompt_template_verificador)
```

```
template_final: str = """Você é responsável pela verificação final das letras de rap.
Seu trabalho é garantir que
a letra revisada esteja dentro dos padrões aceitáveis.
```

Sua resposta final deve ser no formato de dicionário Python:

letra: a letra recebida

Data e hora da verificação: {data_hora_verificacao}

Verificada por um humano: {verificada_por_humano}

Aqui está a letra revisada:

```
{letra_verificada}\
```

```
''''''
```

```
# Criando o template de prompt para a revisão final
```

```
prompt_template_final: PromptTemplate = PromptTemplate(
    input_variables=["letra_verificada", "data_hora_verificacao",
"verificada_por_humano"],
    template=template_final
)
```

```
cadeia_final: LLMChain = LLMChain(
    llm=llm, output_key="saida_final", prompt=prompt_template_final)
```

```
cadeia_sequencial: SequentialChain = SequentialChain(
    memory=SimpleMemory(memories={
        "data_hora_verificacao": str(datetime.utcnow()),
        "verificada_por_humano": "Falso"}),
    chains=[cadeia_rapper, cadeia_verificador, cadeia_final],
    input_variables=["input"],
    output_variables=["saida_final"],
    verbose=True)

resultado = cadeia_sequencial.run(input="violência nas ruas")
print(resultado)
```

Já a **Router Chain** permite realizar roteamento de prompts com base na entrada do usuário. Esse tipo de encadeamento é útil para lidar com diferentes tipos de perguntas ou resolver problemas distintos, enviando-os para o processo correto. É uma técnica poderosa quando sua aplicação precisa lidar com fluxos mais dinâmicos e variados.

Exemplo:

```
from operator import itemgetter
from typing import Literal
from typing_extensions import TypedDict
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableLambda
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini")

prompt_acoes = ChatPromptTemplate.from_messages(
```



```

[
    ("system", "Você é um especialista em investimentos em ações."),
    ("human", "{query}"),
]
)

prompt_renda_fixa = ChatPromptTemplate.from_messages(
    [
        ("system", "Você é um especialista em investimentos de renda fixa."),
        ("human", "{query}"),
    ]
)

chain_acoes = prompt_acoes | llm | StrOutputParser()
chain_renda_fixa = prompt_renda_fixa | llm | StrOutputParser()

route_system = "Roteie a pergunta do usuário para o especialista em ações ou
renda fixa."
route_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", route_system),
        ("human", "{query}"),
    ]
)

class RouteQuery(TypedDict):
    destination: Literal["acoes", "renda_fixa"]

route_chain = (
    route_prompt
    | llm.with_structured_output(RouteQuery)
    | itemgetter("destination")
)

```

```

chain = {
    "destination": route_chain,
    "query": lambda x: x["query"],
} | RunnableLambda(
    lambda x: chain_acoes if x["destination"] == "acoes" else chain_renda_fixa,
)

resultado = chain.invoke({"query": "Quais são os riscos de investir em ações de tecnologia?"})
print(resultado)

```

Este código realiza o roteamento de perguntas sobre investimentos, decidindo se a consulta deve ser respondida por um especialista em **ações** ou **renda fixa**.

1. Modelos de especialistas:

- Um especialista responde perguntas sobre **ações**;
- Outro responde sobre **renda fixa**.

2. Roteamento:

- Um prompt decide para qual especialista a pergunta deve ser enviada com base no conteúdo da pergunta;
- Se a pergunta for sobre "ações", ela vai para o especialista em ações; caso contrário, para o especialista em renda fixa.

3. Execução:

- O código processa a pergunta "Quais são os riscos de investir em ações de tecnologia?";
- Dependendo da resposta do roteador, a pergunta é enviada ao especialista correto e uma resposta é retornada.

Este fluxo permite roteamento simples para perguntas financeiras baseadas no conteúdo da consulta.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, aprendemos sobre **Chains** no LangChain, que são fluxos de trabalho sequenciais usados para processar consultas em várias etapas. Exploramos como criar **prompts customizados**, como roteá-los com base no conteúdo da pergunta e como combinar diferentes **cadeias (chains)** para fornecer respostas precisas e adequadas.



REFERÊNCIAS

LANGCHAIN.chains.router.base.RouterChain. **langchain**. 2024. Disponível em: <https://api.python.langchain.com/en/latest/chains/langchain.chains.router.base.RouterChain.html>. Acesso em: 09 set. 2024.

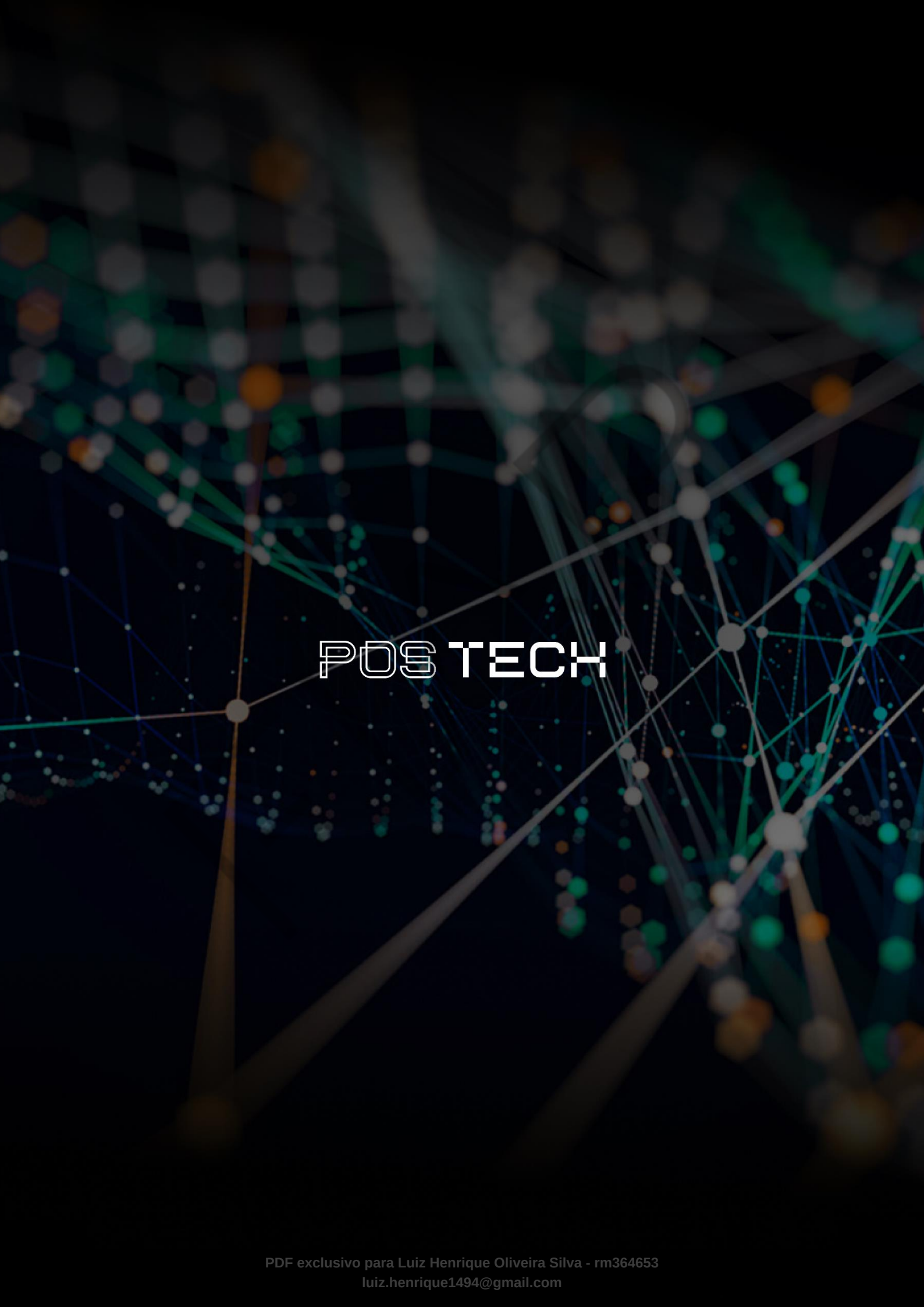
LANGCHAIN.chains.router.llm_router.LLMRouterChain. **langchain**. 2024. Disponível em: https://api.python.langchain.com/en/latest/chains/langchain.chains.router.llm_router.LLMRouterChain.html. Acesso em: 09 set. 2024.

REFLECTIONS on IA. LangChain's Router Chains and Callbacks. **Medium**. 2023. Disponível em: <https://medium.com/@gil.fernandes/langchains-router-chains-and-callbacks-722524c4aa42>. Acesso em: 09 set. 2024.

PALAVRAS-CHAVE

Palavras-chave: RouterChain. SequentialChain. LangChain.

EMENDAS



POSTECH